

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНОМУ УНІВЕРСИТЕТІ “ЛЬВІВСЬКА
ПОЛІТЕХНІКА”**

Кафедра систем штучного інтелекту

Лабораторна робота №3
з дисципліни
“Середовище та компоненти у моделюванні та аналізі даних”

Виконав:
ст. гр. ШІ-31
Хомишин Д.М.

Викладач:
Бенч А. Я.

Тема: Контейнеризація модулів проєкту за допомогою Docker

Мета: Опанувати базові принципи контейнеризації застосунків та навчитися запускати окремі модулі проєкту в Docker-середовищі. Закріпити навички побудови багатомодульного проєкту, роботи з Dockerfile, Docker Compose, томами, мережами та контейнерною взаємодією між сервісами.

Результати

Посилання на репозиторій: <https://github.com/DanyloShi/open-data-ai-analytics>

1. Перебудовано структуру

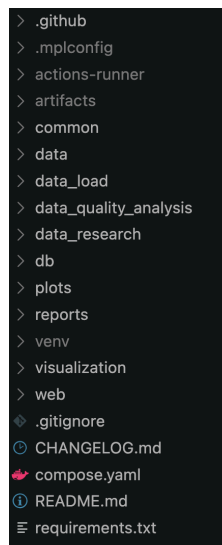


Рис. 1. Структура проєкту

Було перебудовано структуру проєкту для дотримання вимог лабораторної та розділено модулі на окремі папки.

2. Для кожного сервісу додано [app.py](#), DockerFile та requirements.txt

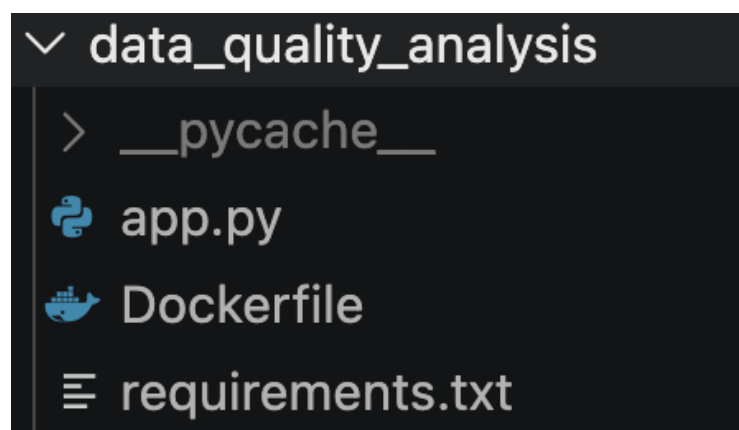


Рис. 2. Структура сервісу

Приклад DockerFile для модуля data_quality_analysis:

```
FROM python:3.12-slim
WORKDIR /app
COPY data_quality_analysis/requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r /app/requirements.txt
COPY common /app/common
COPY data_quality_analysis/app.py /app/data_quality_analysis/app.py
CMD ["python", "data_quality_analysis/app.py"]
```

3. Також було організовано спільний запуск сервісів через docker-compose

```
name: air_quality_pipeline

services:
  data_load:
    build:
      context: .
      dockerfile: data_load/Dockerfile
    environment:
      DB_PATH: /work/db/air_quality.db
      REPORTS_DIR: /work/reports
      PLOTS_DIR: /work/plots
      CSV_PATH: /app/data/dataset.csv
    volumes:
      - pipeline_data:/work
      - ./data:/app/data:ro
    networks:
      - pipeline_net

  data_quality_analysis:
    build:
      context: .
      dockerfile: data_quality_analysis/Dockerfile
    environment:
      DB_PATH: /work/db/air_quality.db
      REPORTS_DIR: /work/reports
      PLOTS_DIR: /work/plots
    volumes:
      - pipeline_data:/work
    depends_on:
      - data_load
    networks:
      - pipeline_net

  data_research:
    build:
      context: .
```

```
    dockerfile: data_research/Dockerfile
environment:
    DB_PATH: /work/db/air_quality.db
    REPORTS_DIR: /work/reports
    PLOTS_DIR: /work/plots
volumes:
    - pipeline_data:/work
depends_on:
    - data_load
networks:
    - pipeline_net

visualization:
    build:
        context: .
        dockerfile: visualization/Dockerfile
    environment:
        DB_PATH: /work/db/air_quality.db
        REPORTS_DIR: /work/reports
        PLOTS_DIR: /work/plots
    volumes:
        - pipeline_data:/work
    depends_on:
        - data_load
    networks:
        - pipeline_net

web:
    build:
        context: .
        dockerfile: web/Dockerfile
    environment:
        REPORTS_DIR: /work/reports
        PLOTS_DIR: /work/plots
        PORT: "8080"
    volumes:
        - pipeline_data:/work:ro
    depends_on:
        - data_quality_analysis
        - data_research
        - visualization
    ports:
        - "8080:8080"
    networks:
        - pipeline_net

volumes:
```

```

pipeline_data:

networks:
  pipeline_net:
    driver: bridge

```

4. Наступним було реалізовано модуль `data_load`, який зчитує CVS-файл, створює таблицю в БД, імпортує дані до БД

```

from __future__ import annotations

import csv
import json
import sqlite3
import sys
from datetime import datetime, timedelta
from pathlib import Path

sys.path.append(str(Path(__file__).resolve().parents[1]))

from common.pipeline_config import CSV_PATH, DB_PATH, REPORTS_DIR, ensure_directories

def _generate_sample_csv(csv_path: Path) -> None:
    csv_path.parent.mkdir(parents=True, exist_ok=True)
    base = datetime(2024, 1, 1, 0, 0, 0)
    rows = []
    for i in range(48):
        rows.append(
            {
                "city_id": str((i % 4) + 1),
                "aqi": round(25 + (i % 20) * 1.8, 2),
                "pm25": round(5 + (i % 18) * 1.3, 2),
                "logged_at": (base + timedelta(hours=i)).strftime("%Y-%m-%d %H:%M:%S"),
            }
        )

    with csv_path.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.DictWriter(fh, fieldnames=["city_id", "aqi", "pm25", "logged_at"])
        writer.writeheader()
        writer.writerows(rows)

def _safe_float(value: str) -> float | None:
    value = (value or "").strip()
    if value == "":
        return None
    try:

```

```

        return float(value)
    except ValueError:
        return None

def _safe_text(value: str) -> str | None:
    value = (value or "").strip()
    return value or None

def load_csv_to_sqlite() -> dict:
    ensure_directories()
    if not CSV_PATH.exists():
        _generate_sample_csv(CSV_PATH)

    with sqlite3.connect(DB_PATH) as conn:
        conn.execute(
            """
            CREATE TABLE IF NOT EXISTS air_quality (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                city_id TEXT,
                aqi REAL,
                pm25 REAL,
                logged_at TEXT
            )
            """
        )
        conn.execute("DELETE FROM air_quality")

        inserted = 0
        with CSV_PATH.open("r", encoding="utf-8") as fh:
            reader = csv.DictReader(fh)
            for row in reader:
                conn.execute(
                    "INSERT INTO air_quality (city_id, aqi, pm25, logged_at) VALUES (?, ?, ?, ?)",
                    (
                        _safe_text(row.get("city_id", "")),
                        _safe_float(row.get("aqi", "")),
                        _safe_float(row.get("pm25", "")),
                        _safe_text(row.get("logged_at", "")),
                    ),
                )
                inserted += 1

        conn.commit()

```

```

summary = {
    "csv_path": str(CSV_PATH),
    "db_path": str(DB_PATH),
    "rows_inserted": inserted,
}

REPORTS_DIR.mkdir(parents=True, exist_ok=True)
(REPORTS_DIR / "load_summary.json").write_text(
    json.dumps(summary, indent=2, ensure_ascii=False),
    encoding="utf-8",
)
(REPORTS_DIR / "load_summary.txt").write_text(
    f"Rows inserted: {inserted}\nDB: {DB_PATH}\nCSV: {CSV_PATH}\n",
    encoding="utf-8",
)

return summary

if __name__ == "__main__":
    result = load_csv_to_sqlite()
    print(json.dumps(result, indent=2, ensure_ascii=False))

```

5. Для візуалізації було створено веб-інтерфейс

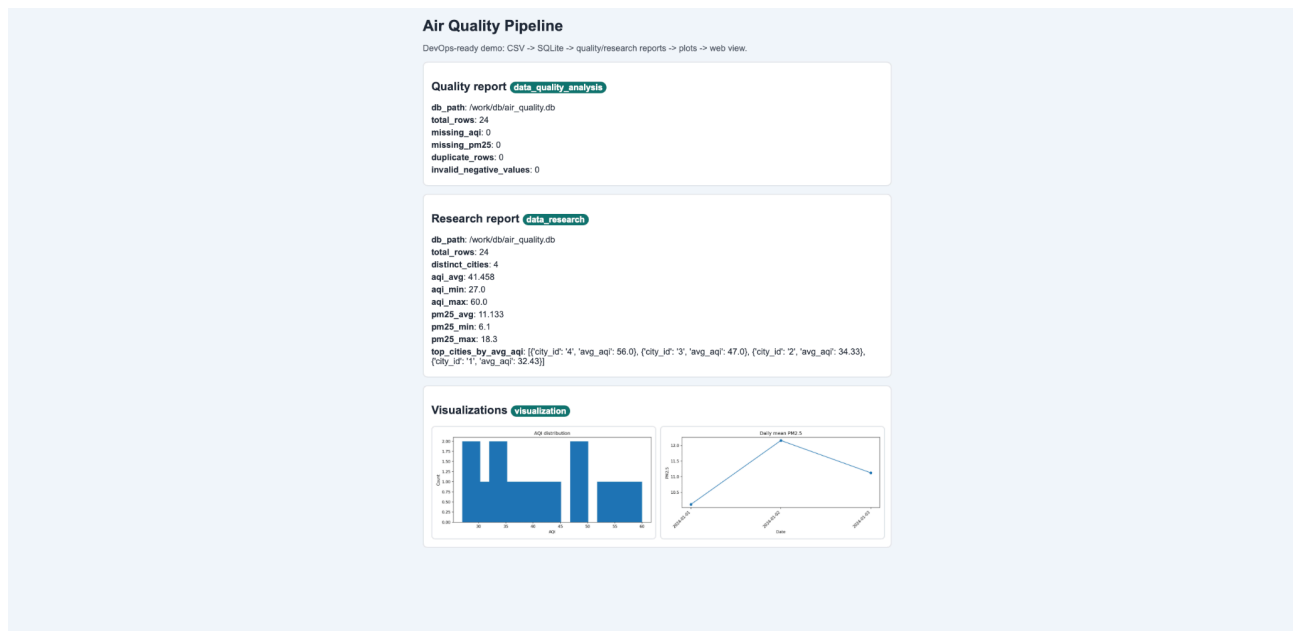


Рис. 3. Веб-інтерфейс

6. Запуск проекту однією командою docker compose up --build

```
(venv) danylo@MacBook-Air-Danylo Середовища та компоненти розробки у моделюванні та аналізі даних % DOCKER_BUILDKIT=0 docker compose up
web-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
web-1 | * Running on all addresses (0.0.0.0)
web-1 | * Running on http://127.0.0.1:8080
web-1 | * Running on http://172.18.0.5:8080
web-1 | Press CTRL+C to quit
visualization-1 | {
visualization-1 |   "plots": [
visualization-1 |     "aqi_hist.png",
visualization-1 |     "pm25_daily_mean.png"
visualization-1 |   ],
visualization-1 |   "rows_used": 24,
visualization-1 |   "aqi_points": 24
visualization-1 | }
visualization-1 exited with code 0
web-1 | 192.168.65.1 - - [15/Apr/2026 20:41:26] "GET / HTTP/1.1" 200 -
web-1 | 192.168.65.1 - - [15/Apr/2026 20:41:27] "GET /plots/aqi_hist.png HTTP/1.1" 200 -
web-1 | 192.168.65.1 - - [15/Apr/2026 20:41:27] "GET /plots/pm25_daily_mean.png HTTP/1.1" 200 -
web-1 | 192.168.65.1 - - [15/Apr/2026 20:41:27] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 -
web-1 | 192.168.65.1 - - [15/Apr/2026 20:41:27] "GET /apple-touch-icon.png HTTP/1.1" 404 -
web-1 | 192.168.65.1 - - [15/Apr/2026 20:41:27] "GET /favicon.ico HTTP/1.1" 404 -
web-1 | 192.168.65.1 - - [15/Apr/2026 20:41:27] "GET /favicon.ico HTTP/1.1" 404 -
```

Рис. 4. Команда docker compose up --build

Висновок

У ході роботи було опановано принципи контейнеризації застосунків за допомогою Docker та виконано поділ проєкту на окремі модулі з власними Dockerfile. Організовано взаємодію сервісів через Docker Compose, налаштовано мережу та спільні томи для обміну даними.

Реалізовано повний цикл обробки даних — від завантаження до аналізу, візуалізації та відображення результатів у веб-інтерфейсі. Отримано практичні навички роботи з Docker та розгортання багатосервісних систем.